ESPE UNIVERSIDAD DE LAS FUERZAS ARMADAS INNOVACIÓN PARA LA EXCELENCIA	INFORME DE PRÁCTICA DE LABORATORIO

DEPARTAMENTO:	ESPE Sede Santo Domingo	**CARRERA:**	Tecnologías de la Información y Comunicación		
ASIGNATURA:	Fundamentos de Programación	**NIVEL:**	1	**FECHA:**	29/10/2026
DOCENTE:	Ing. Esteban Gonzabay Jimenez Mgtr.	**PRÁCTICA N°:**	2	**CALIFICACIÓN:**	

Laboratorio 2: Estructuras repetitivas y anidamiento de estructuras de control en Java Unidad 1, tema 1.7 · NRC 37712

Randy Zambrano

RESUMEN

En este laboratorio se resolvieron cinco ejercicios en Java para practicar las estructuras repetitivas for, while y do-while combinadas con if, else if y switch. Se construyó una tabla de multiplicar de 10 × 10 con ciclos for anidados, un procesador de lecturas de un sensor con valor centinela que evita la división para cero, un juego de adivinanza limitado a siete intentos, un rombo de asteriscos con validación de la entrada y un cajero automático con menú repetitivo. Cada programa se compiló sin advertencias y se ejecutó con los datos de prueba de la guía y con casos propios, incluidos los casos límite: ninguna lectura, último intento permitido, valores inválidos y retiros mayores al saldo. Además, se reprodujeron de forma controlada cinco errores típicos para comprender su efecto y su corrección. Los resultados coincidieron con lo esperado en todos los casos. Se concluye que elegir el ciclo según el problema (for si se conoce la cantidad de repeticiones, while con centinela y do-while para validar o repetir menús) vuelve el código más claro y evita ciclos infinitos y divisiones para cero.

Palabras Claves: ciclos repetitivos, anidamiento, Java.

1. **INTRODUCCIÓN:**

En la asignatura Fundamentos de Programación, esta práctica cierra la Unidad 1 con el uso de las estructuras repetitivas for, while y do-while, combinadas con decisiones (if, else if, switch) y con ciclos anidados. Estas estructuras permiten que un programa procese series de datos, valide lo que escribe el usuario y ofrezca menús que se repiten hasta que la persona decida salir.

El propósito del laboratorio no es solo que los programas funcionen, sino justificar qué ciclo conviene en cada caso y comprobar su comportamiento en los casos límite (sin datos, valores inválidos, último intento). Para ello se siguió la disciplina de trabajo indicada en la guía: un archivo por ejercicio con nombre Lab2_E#_Zambrano, encabezado con nombre, NRC y fecha, variables descriptivas, comentarios que explican el proceso, compilación sin errores ni advertencias y pruebas con datos de la guía y con casos propios.

2. **OBJETIVO(S):**

2.1 **Objetivo general:** implementar programas en Java que utilicen estructuras repetitivas y estructuras de control anidadas para procesar series de datos, validar entradas y construir menús, eligiendo de forma justificada el ciclo adecuado para cada problema.

2.2 Construir ciclos for anidados para generar tablas y patrones con salida alineada.

2.3 Procesar series de datos de longitud desconocida con while y un valor centinela, usando contadores y acumuladores.

2.4 Validar entradas del usuario y construir menús repetitivos con do-while y switch.

2.5 Comprobar el comportamiento de los programas en casos límite: sin datos, valores inválidos y último intento.

3. **MARCO TEÓRICO:**

3.1 Estructuras repetitivas en Java

Una estructura repetitiva ejecuta un bloque de instrucciones mientras una condición lógica sea verdadera. Java ofrece tres formas principales:

- **for:** reúne inicialización, condición y actualización en una sola línea. Es la opción natural cuando el número de repeticiones se conoce de antemano.

- **while:** evalúa la condición antes de entrar al bloque, por lo que puede no ejecutarse nunca. Es adecuado cuando la repetición depende de un valor centinela.
- **do-while:** ejecuta el bloque y evalúa la condición al final, por lo que se ejecuta al menos una vez. Es adecuado para validar datos y para menús.

```
for (int i = 1; i <= n; i++) { ... }
while (condicion) { ... }          // puede no ejecutarse nunca
do { ... } while (condicion);      // se ejecuta al menos una vez
```

3.2 Criterio para elegir el ciclo

¿Se conoce el número de repeticiones?	¿El bloque debe ejecutarse al menos una vez?	Ciclo	Uso en este laboratorio
Sí	—	for	E1 (tabla 10 × 10) y filas del rombo (E4)
No	No	while	E2 (lecturas hasta un centinela)
No	Sí	do-while	E3 (intentos), E4 (validación) y E5 (menú)

3.3 Ciclos anidados

Un ciclo anidado es un ciclo dentro de otro. Por cada vuelta del ciclo externo, el interno completa todas sus vueltas. Si el externo se repite n veces y el interno m veces, la instrucción más interna se ejecuta n · m veces. Se usa para recorrer matrices, generar tablas y dibujar figuras formadas por filas y columnas.

3.4 Contadores, acumuladores y centinela

Un **contador** aumenta de uno en uno y registra cuántas veces ocurre algo; un **acumulador** suma valores distintos en cada vuelta. Ambos se inicializan justo antes del ciclo. Un **valor centinela** es un dato especial (aquí, cualquier número negativo) que indica el fin de la serie y no forma parte de los datos. Cuando el ciclo puede terminar sin haber procesado datos, hay que proteger las operaciones que dividen por el contador para evitar una división para cero.

3.5 Decisiones dentro de los ciclos y salida con formato

Dentro de un ciclo se usan if / else if para decidir según el dato y switch para elegir entre opciones de un menú; cada case termina con break para que no se ejecuten también los casos siguientes. La salida alineada se logra con `System.out.printf("%4d", valor)`, que reserva un ancho fijo de 4 caracteres y alinea el número a la derecha.

4. DESCRIPCIÓN DEL PROCEDIMIENTO:

4.1 Materiales y entorno

- Java Development Kit 21 LTS, que incluye el compilador javac y la máquina virtual (JVM).
- Compilación y ejecución desde la terminal: `javac -Xlint:all Clase.java` y `java Clase`. El indicador -Xlint:all activa todas las advertencias del compilador.
- Proyecto Zambrano_Randy_Lab2 con una clase pública por ejercicio: Lab2_E1_Zambrano, Lab2_E2_Zambrano, Lab2_E3_Zambrano, Lab2_E4_Zambrano y Lab2_E5_Zambrano.
- Guía de laboratorio VDC-GUI-2024-V2-015 y programas de ejemplo de la clase. Reactivos: no aplica.

4.2 Método de trabajo

Para cada ejercicio se siguieron los mismos pasos: (1) analizar entradas, proceso y salidas; (2) definir la condición de salida del ciclo y verificar que se alcanza; (3) escribir el pseudocódigo; (4) programar con nombres descriptivos y comentarios; (5) compilar hasta no tener errores ni advertencias; (6) ejecutar con los datos de prueba de la guía y con al menos un caso propio, comparando la salida con lo esperado; (7) registrar capturas y errores encontrados.

4.3 E1 · Tabla de multiplicar completa (1,5 pts)

Enunciado: Imprimir la tabla de multiplicar del 1 al 10 como matriz de 10 filas por 10 columnas, con cada valor alineado a la derecha en un ancho fijo.

Análisis	Descripción
Entradas	Ninguna (los límites 10 × 10 son constantes del programa).
Proceso	Ciclo externo `i` recorre las filas de 1 a 10; ciclo interno `j` recorre las columnas de 1 a 10; se imprime `i * j` con `%4d` y al terminar cada fila se hace un salto de línea. Un contador cuenta las ejecuciones de la instrucción interna.
Salidas	Matriz de 10 filas alineada; última fila termina en 100; mensaje con el total de ejecuciones.
Ciclo elegido	for anidados: se conoce la cantidad de repeticiones.

Pseudocódigo

```
INICIO
  FILAS ← 10; COLUMNAS ← 10; ejecuciones ← 0
  PARA i DESDE 1 HASTA FILAS
    PARA j DESDE 1 HASTA COLUMNAS
```

```
      ESCRIBIR i * j con ancho 4, alineado a la derecha
      ejecuciones ← ejecuciones + 1
    FIN PARA
    SALTO DE LÍNEA
  FIN PARA
  ESCRIBIR ejecuciones
FIN
```

Código fuente comentado (capturas de VS Code: Lab2_E1_Zambrano.java)

J Lab2_E1_Zambrano.java > Lab2_E1_Zambrano

```java
/*
 * Nombre completo: Randy Zambrano
 * NRC: 37712
 * Fecha: 29/10/2026
 * Enunciado: Imprimir la tabla de multiplicar del 1 al 10 como matriz de
 *            10 filas por 10 columnas, con cada valor alineado a la derecha
 *            en un ancho fijo, usando dos ciclos for anidados.
 */
public class Lab2_E1_Zambrano {

    Run | Debug
    public static void main(String[] args) {
        final int FILAS = 10;      // cantidad de filas (multiplicando)
        final int COLUMNAS = 10;   // cantidad de columnas (multiplicador)
        int ejecuciones = 0;       // cuenta cuántas veces corre la instrucción interna

        System.out.println("TABLA DE MULTIPLICAR (" + FILAS + " x " + COLUMNAS + ")");

        // Ciclo externo: una vuelta por cada fila de la matriz
        for (int i = 1; i <= FILAS; i++) {
            // Ciclo interno: completa todas las columnas de la fila actual
            for (int j = 1; j <= COLUMNAS; j++) {
                System.out.printf("%4d", i * j);   // ancho fijo de 4, alineado a la derecha
                ejecuciones++;
            }
            System.out.println();                 // salto de línea al terminar la fila
        }

```

Figura 1. *Código de Lab2_E1_Zambrano.java, parte 1 de 2.*

J Lab2_E1_Zambrano.java > Lab2_E1_Zambrano

```java
        System.out.println("Instrucción interna ejecutada " + ejecuciones + " veces.");
    }
}
```

Figura 2. *Código de Lab2_E1_Zambrano.java, parte 2 de 2.*

4.4 E2 · Estadística de un sensor (2 pts)

Enunciado: Leer lecturas de temperatura hasta ingresar un valor negativo (centinela) y mostrar cuántas lecturas válidas hubo, el promedio, el máximo y el mínimo con 2 decimales, sin dividir para cero si no hubo lecturas.

Análisis	Descripción
Entradas	Lecturas de temperatura (números reales) hasta un valor negativo.
Proceso	Se lee una primera lectura; mientras sea mayor o igual que 0 se cuenta, se suma y se actualizan máximo y mínimo (la primera lectura válida los inicializa); se lee la siguiente. Al terminar, si hubo lecturas, se calcula suma / cantidad.
Salidas	Cantidad, promedio, máximo y mínimo con 2 decimales; o un aviso si no hubo lecturas.
Ciclo elegido	while con centinela: no se sabe cuántas lecturas habrá y puede haber cero.

Pseudocódigo

```
INICIO
  cantidad ← 0; suma ← 0; maximo ← 0; minimo ← 0
  LEER lectura
  MIENTRAS lectura ≥ 0 HACER
    cantidad ← cantidad + 1
    suma ← suma + lectura
    SI cantidad = 1 ENTONCES
```

```
      maximo ← lectura; minimo ← lectura
    SINO
      SI lectura > maximo ENTONCES maximo ← lectura
      SI lectura < minimo ENTONCES minimo ← lectura
    FIN SI
    LEER lectura
  FIN MIENTRAS
  SI cantidad > 0 ENTONCES
    promedio ← suma / cantidad
    ESCRIBIR cantidad, promedio, maximo, minimo
  SINO
    ESCRIBIR "No hubo lecturas válidas"
  FIN SI
FIN
```

Código fuente comentado (capturas de VS Code: Lab2_E2_Zambrano.java)

J Lab2_E2_Zambrano.java > Lab2_E2_Zambrano

```java
/*
 * Nombre completo: Randy Zambrano
 * NRC: 37712
 * Fecha: 29/10/2026
 * Enunciado: Leer lecturas de temperatura hasta que se ingrese un valor negativo
 *            (centinela). Mostrar cuántas lecturas válidas hubo, el promedio, el
 *            máximo y el mínimo con 2 decimales, sin dividir para cero si no
 *            hubo lecturas.
 */
import java.util.Locale;
import java.util.Scanner;

public class Lab2_E2_Zambrano {

    Run | Debug
    public static void main(String[] args) {
        // Locale.US para que el punto sea el separador decimal (20.5)
        Scanner teclado = new Scanner(System.in).useLocale(Locale.US);

        // Contador y acumulador se inicializan justo antes del ciclo
        int cantidad = 0;
        double suma = 0;
        double maximo = 0;
        double minimo = 0;

```

Figura 3. *Código de Lab2_E2_Zambrano.java, parte 1 de 3.*

J Lab2_E2_Zambrano.java > Lab2_E2_Zambrano

```
25          // Lectura inicial (lectura "cebadora"): permite que el while decida si entra
26          System.out.print("Temperatura (valor negativo para terminar): ");
27          double lectura = teclado.nextDouble();
28
29          while (lectura >= 0) {                // el centinela es cualquier valor negativo
30              cantidad++;
31              suma += lectura;
32
33              if (cantidad == 1) {              // la primera lectura válida inicia max y min
34                  maximo = lectura;
35                  minimo = lectura;
36              } else {
37                  if (lectura > maximo) {
38                      maximo = lectura;
39                  }
40                  if (lectura < minimo) {
41                      minimo = lectura;
42                  }
43              }
44
45              // Se vuelve a leer dentro del ciclo para que la condición pueda cambiar
46              System.out.print("Temperatura (valor negativo para terminar): ");
47              lectura = teclado.nextDouble();
48          }
49
```

Figura 4. *Código de Lab2_E2_Zambrano.java, parte 2 de 3.*

J Lab2_E2_Zambrano.java > Lab2_E2_Zambrano

```
50          // Caso límite: sin lecturas válidas no se calcula el promedio
51          if (cantidad > 0) {
52              double promedio = suma / cantidad;
53              System.out.printf(Locale.US, "Lecturas: %d Promedio: %.2f Max: %.2f Min: %.2f%n",
54                      cantidad, promedio, maximo, minimo);
55          } else {
56              System.out.println("No hubo lecturas válidas: no se muestran estadísticas.");
57          }
58      }
59  }
```

Figura 5. *Código de Lab2_E2_Zambrano.java, parte 3 de 3.*

4.5 E3 · Adivina el número (2 pts)

Enunciado: El programa genera un número aleatorio entre 1 y 100; el usuario tiene 7 intentos y en cada uno se indica «más alto» o «más bajo». Al final se informa si acertó y en cuántos intentos, o cuál era el número.

Análisis	Descripción
Entradas	Números enteros propuestos por el usuario (uno por intento).
Proceso	Se genera `secreto = azar.nextInt(100) + 1`. En cada vuelta se lee el número, se incrementa el contador de intentos y se compara con el secreto (igual → acertó; menor → «más alto»; mayor → «más bajo»). El ciclo continúa mientras no haya acertado y queden intentos.
Salidas	Pista en cada intento; mensaje final de victoria con el número de intentos o de derrota con el número secreto.
Ciclo elegido	do-while: el usuario juega al menos un intento; condición compuesta (no acertó Y quedan intentos).

Pseudocódigo

```
INICIO
  MAX_INTENTOS ← 7
  secreto ← número aleatorio entre 1 y 100
  intentos ← 0; acerto ← FALSO
  REPETIR
    LEER propuesto
    intentos ← intentos + 1
    SI propuesto = secreto ENTONCES acerto ← VERDADERO
    SINO SI propuesto < secreto ENTONCES ESCRIBIR "más alto"
    SINO ESCRIBIR "más bajo"
  HASTA QUE acerto = VERDADERO O intentos = MAX_INTENTOS
```

```
  SI acerto ENTONCES ESCRIBIR "Acertaste en", intentos
  SINO ESCRIBIR "Perdiste. El número era", secreto
FIN
```

Código fuente comentado (capturas de VS Code: Lab2_E3_Zambrano.java)

J Lab2_E3_Zambrano.java > Lab2_E3_Zambrano

```java
/*
 * Nombre completo: Randy Zambrano
 * NRC: 37712
 * Fecha: 29/10/2026
 * Enunciado: Adivina el número. El programa elige un número aleatorio entre
 *            1 y 100; el usuario tiene 7 intentos y en cada uno se indica
 *            "más alto" o "más bajo". Al final se informa si acertó y en
 *            cuántos intentos, o cuál era el número.
 */
import java.util.Random;
import java.util.Scanner;

public class Lab2_E3_Zambrano {

    Run | Debug
    public static void main(String[] args) {
        final int MAX_INTENTOS = 7;
        Scanner teclado = new Scanner(System.in);
        Random azar = new Random();
        int secreto = azar.nextInt(100) + 1;   // número entre 1 y 100

        int intentos = 0;
        boolean acerto = false;

        System.out.println("ADIVINA EL NÚMERO: pensé un número entre 1 y 100.");
        System.out.println("Tienes " + MAX_INTENTOS + " intentos.");

```

Figura 6. *Código de Lab2_E3_Zambrano.java, parte 1 de 2.*

J Lab2_E3_Zambrano.java > Lab2_E3_Zambrano

```java
        // do-while: el usuario siempre juega al menos una vez.
        // Condición compuesta: sigue mientras NO acertó Y le quedan intentos.
        do {
            System.out.print("Intento " + (intentos + 1) + " de " + MAX_INTENTOS + ": ");
            int propuesto = teclado.nextInt();
            intentos++;

            if (propuesto == secreto) {
                acerto = true;
            } else if (propuesto < secreto) {
                System.out.println("  El número secreto es más alto.");
            } else {
                System.out.println("  El número secreto es más bajo.");
            }
        } while (!acerto && intentos < MAX_INTENTOS);

        if (acerto) {
            System.out.println("¡Acertaste en " + intentos + " intento(s)!");
        } else {
            System.out.println("Perdiste. El número era " + secreto + ".");
        }
    }
}
```

Figura 7. *Código de Lab2_E3_Zambrano.java, parte 2 de 2.*

4.6 E4 · Rombo de asteriscos (2 pts)

Enunciado: Leer un número impar n y dibujar un rombo de n filas, validando con do-while que n sea impar y positivo.

Análisis	Descripción
Entradas	Entero n; se repite la lectura mientras no sea impar y positivo.
Proceso	Validación: se pide n y se repite si $n \le 0$ o n es par. Dibujo: `mitad = n / 2`; para cada fila se calcula `k` (la fila en la mitad superior y su reflejo en la inferior), `espacios = mitad − k` y `asteriscos = 2·k + 1`, que se imprimen con dos for anidados.
Salidas	Mensaje de error en entradas inválidas y rombo de n filas.
Ciclo elegido	do-while para validar (siempre se pide al menos una vez) y for anidados para espacios y asteriscos.

Pseudocódigo

```
INICIO
  REPETIR
    LEER n
    SI n ≤ 0 O n es par ENTONCES ESCRIBIR "Valor inválido"
  HASTA QUE n > 0 Y n es impar
  mitad ← n div 2
  PARA fila DESDE 0 HASTA n − 1
    SI fila ≤ mitad ENTONCES k ← fila SINO k ← n − 1 − fila
    espacios ← mitad − k; asteriscos ← 2 * k + 1
    PARA e DESDE 1 HASTA espacios: ESCRIBIR " "
    PARA a DESDE 1 HASTA asteriscos: ESCRIBIR "*"
    SALTO DE LÍNEA
  FIN PARA
FIN
```

Código fuente comentado (capturas de VS Code: Lab2_E4_Zambrano.java)

J Lab2_E4_Zambrano.java > Lab2_E4_Zambrano

```java
/*
 * Nombre completo: Randy Zambrano
 * NRC: 37712
 * Fecha: 29/10/2026
 * Enunciado: Leer un número impar n y dibujar un rombo de asteriscos de n filas.
 *            Validar con do-while que n sea impar y positivo, repitiendo la
 *            lectura si no lo es.
 */
import java.util.Scanner;

public class Lab2_E4_Zambrano {

    Run | Debug
    public static void main(String[] args) {
        Scanner teclado = new Scanner(System.in);
        int n;

        // do-while: hay que pedir el dato al menos una vez antes de validarlo
        do {
            System.out.print("Ingrese un número impar positivo (tamaño del rombo): ");
            n = teclado.nextInt();
            if (n <= 0 || n % 2 == 0) {
                System.out.println("  Valor inválido: debe ser impar y positivo.");
            }
        } while (n <= 0 || n % 2 == 0);

        int mitad = n / 2;                    // índice de la fila central

```

Figura 8. *Código de Lab2_E4_Zambrano.java, parte 1 de 2.*

J Lab2_E4_Zambrano.java > Lab2_E4_Zambrano

```java
        // Ciclo externo: una vuelta por fila
        for (int fila = 0; fila < n; fila++) {
            // Mitad superior (incluye el centro): k = fila; mitad inferior: se refleja
            int k = (fila <= mitad) ? fila : (n - 1 - fila);
            int espacios = mitad - k;          // sangría izquierda
            int asteriscos = 2 * k + 1;        // asteriscos de la fila

            for (int e = 0; e < espacios; e++) {
                System.out.print(" ");
            }
            for (int a = 0; a < asteriscos; a++) {
                System.out.print("*");
            }
            System.out.println();
        }
    }
}
```

Figura 9. *Código de Lab2_E4_Zambrano.java, parte 2 de 2.*

4.7 E5 · Cajero automático (2,5 pts)

Enunciado: Menú repetitivo (do-while + switch) con saldo inicial de $500: 1) consultar, 2) depositar, 3) retirar, 0) salir. No se permiten depósitos ni retiros con montos negativos o cero, ni retiros mayores al saldo. Una opción inexistente muestra un mensaje.

Análisis	Descripción
Entradas	Opción del menú (entero) y monto (real) en depósitos y retiros.
Proceso	Se muestra el menú y se lee la opción; switch elige la acción: consultar muestra el saldo; depositar acepta solo montos > 0; retirar acepta solo montos > 0 y ≤ saldo; default avisa de opción inválida. El ciclo termina con la opción 0.
Salidas	Saldo con 2 decimales y mensajes de aceptación, rechazo, opción inválida o fin del programa.
Ciclo elegido	do-while: el menú debe mostrarse al menos una vez; switch para las opciones.

Pseudocódigo

```
INICIO
  saldo ← 500.00
  REPETIR
    MOSTRAR menú; LEER opcion
    SEGÚN opcion
      1: ESCRIBIR saldo
      2: LEER monto
         SI monto > 0 ENTONCES saldo ← saldo + monto; "Depósito aceptado"
         SINO "Monto inválido"
      3: LEER monto
         SI monto > 0 Y monto ≤ saldo ENTONCES saldo ← saldo − monto; "Retiro aceptado"
         SINO "Fondos insuficientes o monto inválido"
      0: ESCRIBIR "Fin del programa"
      OTRO: ESCRIBIR "Opción inválida"
  HASTA QUE opcion = 0
FIN
```

Código fuente comentado (capturas de VS Code: Lab2_E5_Zambrano.java)

J Lab2_E5_Zambrano.java > Lab2_E5_Zambrano

```java
/*
 * Nombre completo: Randy Zambrano
 * NRC: 37712
 * Fecha: 29/10/2026
 * Enunciado: Cajero automático con menú repetitivo (do-while + switch) y saldo
 *            inicial de $500: 1) consultar, 2) depositar, 3) retirar, 0) salir.
 *            No se permiten depósitos ni retiros con montos negativos o cero,
 *            ni retiros mayores que el saldo. Una opción inexistente muestra
 *            un mensaje.
 */
import java.util.Locale;
import java.util.Scanner;

public class Lab2_E5_Zambrano {

    Run | Debug
    public static void main(String[] args) {
        Scanner teclado = new Scanner(System.in).useLocale(Locale.US);
        double saldo = 500.00;
        int opcion;

```

Figura 10. *Código de Lab2_E5_Zambrano.java, parte 1 de 3.*

J Lab2_E5_Zambrano.java > Lab2_E5_Zambrano

```java
        // do-while: el menú se muestra al menos una vez y se repite hasta elegir 0
        do {
            System.out.println();
            System.out.println("--- CAJERO AUTOMÁTICO ---");
            System.out.println("1) Consultar  2) Depositar  3) Retirar  0) Salir");
            System.out.print("Opción: ");
            opcion = teclado.nextInt();

            switch (opcion) {
                case 1:
                    System.out.printf(Locale.US, "Saldo: $%.2f%n", saldo);
                    break;
                case 2:
                    System.out.print("Monto a depositar: ");
                    double deposito = teclado.nextDouble();
                    if (deposito > 0) {
                        saldo += deposito;
                        System.out.println("Depósito aceptado");
                    } else {
                        System.out.println("Monto inválido");
                    }
                    break;
                case 3:
                    System.out.print("Monto a retirar: ");
                    double retiro = teclado.nextDouble();
                    if (retiro > 0 && retiro <= saldo) {
                        saldo -= retiro;
```

Figura 11. *Código de Lab2_E5_Zambrano.java, parte 2 de 3.*

J Lab2_E5_Zambrano.java > Lab2_E5_Zambrano

```
48                          System.out.println("Retiro aceptado");
49                      } else {
50                          System.out.println("Fondos insuficientes o monto inválido");
51                      }
52                      break;
53                  case 0:
54                      System.out.println("Fin del programa");
55                      break;
56                  default:
57                      System.out.println("Opción inválida");
58              }
59          } while (opcion != 0);
60      }
61  }
```

Figura 12. *Código de Lab2_E5_Zambrano.java, parte 3 de 3.*

4.8 Registro del uso de inteligencia artificial

Siguiendo la instrucción 10 de la guía se declara el uso de una herramienta de inteligencia artificial: **Claude (Anthropic)** se utilizó como apoyo para preparar un borrador de los programas, del pseudocódigo y de la estructura del informe. Los programas se compilaron con `javac -Xlint:all` en JDK 21 (sin errores ni advertencias) y se ejecutaron con todos los datos de prueba de la guía y con casos propios, comparando cada salida con el resultado esperado. Los errores de la sección 5.5 se reprodujeron ejecutando variantes del código. Las capturas de la sección 6 corresponden a esas ejecuciones.

5. ANÁLISIS DE RESULTADOS:

5.1 Resultados por ejercicio

E1 · Tabla de multiplicar

Verificación	Esperado	Obtenido	Estado
Primera fila	1 2 3 … 10	1 2 3 … 10	Correcto
Última fila	Termina en 100	10 20 … 90 100	Correcto
Alineación	Columnas alineadas a la derecha	Ancho fijo de 4 con % 4d	Correcto
Ejecuciones de la instrucción interna	10 × 10 = 100	100	Correcto
Variante 5 × 7 (cambiando constantes)	5 × 7 = 35	35	Correcto

E2 · Estadística de un sensor

Entrada	Salida esperada	Salida obtenida	Estado
20.5 22 19.5 -1	Lecturas: 3 Promedio: 20.67 Max: 22.00 Min: 19.50	Lecturas: 3 Promedio: 20.67 Max: 22.00 Min: 19.50	Correcto
-5	No muestra estadísticas	No hubo lecturas válidas: no se muestran estadísticas.	Correcto
0 35.2 18 18 27.75 -3 (propio)	Lecturas: 5 Promedio: 19.79 Max: 35.20 Min: 0.00	Lecturas: 5 Promedio: 19.79 Max: 35.20 Min: 0.00	Correcto
7.5 -1 (propio, una sola lectura)	Lecturas: 1 Promedio: 7.50 Max: 7.50 Min: 7.50	Lecturas: 1 Promedio: 7.50 Max: 7.50 Min: 7.50	Correcto

El caso propio con una lectura de 0 comprueba que el cero es una lectura válida (solo los negativos son centinela) y que el mínimo se inicializa con la primera lectura y no con 0.

E3 · Adivina el número

Caso	Intentos ingresados	Resultado obtenido	Estado
Partida ganada (secreto 37)	50, 25, 37	¡Acertaste en 3 intento(s)!	Correcto
Partida perdida (secreto 35)	10, 90, 20, 80, 30, 40, 50	Perdiste. El número era 35.	Correcto
Último intento permitido (secreto 100)	50, 75, 88, 94, 97, 99, 100	¡Acertaste en 7 intento(s)!	Correcto

E4 · Rombo de asteriscos

Entrada	Esperado	Obtenido	Estado
4, 0, 5	Rechaza 4 y 0; con 5 dibuja un rombo de 5 filas	Rechazó 4 y 0; dibujó 1, 3, 5, 3, 1 asteriscos	Correcto
-3, 9 (propio)	Rechaza -3; con 9 dibuja un rombo de 9 filas	Rechazó -3; dibujó 1, 3, 5, 7, 9, 7, 5, 3, 1 asteriscos	Correcto
1 (caso límite)	Un solo asterisco	*	Correcto

E5 · Cajero automático (secuencia de la guía)

Secuencia de prueba	Resultado esperado	Resultado obtenido	Saldo luego	Estado
1	Saldo: $500.00	Saldo: $500.00	500.00	Correcto
2 → 150	Depósito aceptado	Depósito aceptado	650.00	Correcto
3 → 800	Fondos insuficientes o monto inválido	Fondos insuficientes o monto inválido	650.00	Correcto
3 → -20	Fondos insuficientes o monto inválido	Fondos insuficientes o monto inválido	650.00	Correcto
3 → 100, luego 1	Saldo: $550.00	Retiro aceptado; Saldo: $550.00	550.00	Correcto
9	Opción inválida	Opción inválida	550.00	Correcto
0	Fin del programa	Fin del programa	550.00	Correcto

Caso propio: con saldo $500.00 se probaron un retiro de 0 (rechazado), un depósito de 0 (rechazado), un retiro exacto de 500 (aceptado, saldo $0.00), un retiro de 1 con saldo 0 (rechazado) y la opción 7 (inválida).

5.2 Pruebas de escritorio

Prueba de escritorio de E2 con la entrada 20.5, 22, 19.5, -1

Paso	lectura	lectura ≥ 0	cantidad	suma	maximo	minimo
Inicio	—	—	0	0.0	0.0	0.0
Lee 20.5	20.5	Verdadero	1	20.5	20.5	20.5

Paso	lectura	lectura ≥ 0	cantidad	suma	maximo	minimo
Lee 22	22.0	Verdadero	2	42.5	22.0	20.5
Lee 19.5	19.5	Verdadero	3	62.0	22.0	19.5
Lee -1	-1.0	Falso → sale del ciclo	3	62.0	22.0	19.5
Salida	cantidad > 0 → promedio = 62.0 / 3 = 20.67		3	62.0	22.00	19.50

Prueba de escritorio de E4: validación (4, 0, 5) y dibujo con n = 5

Lectura	n ≤ 0	n % 2 == 0	¿Repite?
4	Falso	Verdadero	Sí: «Valor inválido»
0	Verdadero	Verdadero	Sí: «Valor inválido»
5	Falso	Falso	No: sale del do-while

Con n = 5 resulta `mitad = 5 / 2 = 2`.

fila	k	espacios = mitad − k	asteriscos = 2·k + 1	Línea impresa
0	0	2	1	*
1	1	1	3	***
2	2	0	5	*
3	5 − 1 − 3 = 1	1	3	***
4	5 − 1 − 4 = 0	2	1	*

5.3 Análisis de E3: por qué la búsqueda binaria acierta en 7 intentos o menos

En la búsqueda binaria se propone siempre el número central del rango que aún es posible. Si no es el correcto, la pista («más alto» o «más bajo») descarta la mitad de las opciones. En el peor caso, la cantidad de candidatos restantes antes de cada intento es:

Intento	Candidatos posibles antes del intento (peor caso)	Candidatos tras fallar
1	100	50
2	50	25
3	25	12
4	12	6
5	6	3
6	3	1
7	1	0 (el intento 7 acierta)

En general, con k intentos se pueden distinguir hasta 2^k − 1 números. Como 2^6 − 1 = 63 < 100 ≤ 127 = 2^7 − 1, bastan 7 intentos, es decir, $\lceil \log_2(100+1) \rceil = 7$. Esto se comprobó con el secreto 100 (peor caso para esa secuencia), que se acertó justo en el intento 7.

5.4 Respuestas a las preguntas de análisis

1. ¿Qué ciclo utilizó en cada ejercicio y por qué era el más adecuado?

Ejercicio	Ciclo	Justificación
E1	for anidados	Se conoce de antemano el número de filas y columnas (10 y 10).
E2	while con centinela	No se sabe cuántas lecturas habrá y puede haber cero; el while evalúa antes de entrar y permite no ejecutar nunca el bloque.
E3	do-while	El usuario siempre hace al menos un intento; la condición compuesta (no acertó y quedan intentos) se evalúa al final de cada vuelta.
E4	do-while + for anidados	La entrada se pide al menos una vez antes de validarla (do-while); las filas, espacios y asteriscos tienen cantidad conocida (for).
E5	do-while + switch	El menú debe mostrarse al menos una vez y repetirse hasta elegir 0; switch selecciona la acción.

2. ¿Cuántas veces se ejecuta la instrucción interna de dos ciclos for anidados de 1 a 10? Generalice para n y m.

Se ejecuta 10 × 10 = **100** veces: el ciclo externo da 10 vueltas y, en cada una, el interno da 10. En general, si el externo repite n veces y el interno m veces, la instrucción interna se ejecuta **n · m** veces. Se verificó con el contador del programa (100 para 10 × 10 y 35 para 5 × 7). Con tres ciclos anidados de n, m y p vueltas serían n · m · p.

3. ¿Qué ocurre en E2 si no se controla el caso sin lecturas? ¿Qué tipo de error produce?

Si la primera lectura es negativa, `cantidad` vale 0 y el promedio se calcularía como `suma / cantidad`, es decir, 0 / 0. Con variables enteras Java lanza una excepción en tiempo de ejecución, `ArithmeticException: / by zero` (Figura 30). Con variables `double`, como en este programa, no lanza excepción: devuelve `NaN` y el programa muestra «Promedio: NaN» junto con un máximo y un mínimo de 0.00 que no corresponden a ninguna lectura (Figura 29). En ambos casos es un error en tiempo de ejecución o de lógica que se evita con `if (cantidad > 0)`.

4. ¿Qué diferencia práctica existe entre validar una entrada con while y con do-while?

El while evalúa la condición antes de ejecutar el bloque, así que obliga a pedir y leer el dato una vez antes del ciclo y repetir esas mismas instrucciones dentro de él. El do-while ejecuta primero y evalúa después, por lo que el dato se pide una sola vez en el código y siempre se pide al menos una vez. Con while, la validación del rombo quedaría así:

J Validacion_con_while.java > main

```
1  System.out.print("Ingrese un número impar positivo: ");
2  n = teclado.nextInt();                          // lectura duplicada
3  while (n <= 0 || n % 2 == 0) {
4      System.out.println("Valor inválido");
5      System.out.print("Ingrese un número impar positivo: ");
6      n = teclado.nextInt();                      // lectura repetida
7  }
```

Figura 13. *Validación del rombo (E4) escrita con while: la lectura se repite dos veces.*

Ambas formas son correctas, pero el do-while evita duplicar código y por eso es la opción más clara para validar y para menús.

5.5 Registro de errores encontrados y correcciones aplicadas

Los cinco programas compilaron con `javac -Xlint:all` sin errores ni advertencias. Para comprender los «Problemas frecuentes» de la guía, los siguientes errores se **reprodujeron de forma controlada** modificando copias del código y se ejecutaron para observar su efecto.

N.º	Error reproducido	Hipótesis	Corrección aplicada	Evidencia
1	E2: mínimo inicializado en 0 (sin usar la primera lectura)	Como ninguna temperatura positiva es menor que 0, la condición `lectura < minimo` nunca se cumple.	Inicializar máximo y mínimo con la primera lectura válida (`if (cantidad == 1)`).	Fig. 28: Min: 0.00 en lugar de 19.50
2	E2: promedio sin controlar cantidad = 0 (double)	0.0 / 0 no es una excepción en `double`, produce un valor no numérico.	Mostrar estadísticas solo si `cantidad > 0`.	Fig. 29: Promedio: NaN
3	Promedio con enteros y cantidad = 0	La división entera entre cero sí lanza excepción.	Mismo control `if (cantidad > 0)`.	Fig. 30: ArithmeticException: / by zero
4	E5: falta `break` en `case 1`	Al faltar el break, la ejecución «cae» al `case 2` y ejecuta también el depósito.	Agregar `break;` al final de cada case.	Fig. 31: tras consultar el saldo pide «Monto a depositar»
5	E3: condición `intentos <= MAX_INTENTOS`	El operador <= permite un intento más (error de uno).	Usar `intentos < MAX_INTENTOS`.	Fig. 32: aparece «Intento 8 de 7»

Otro riesgo conocido es el ciclo infinito (variable de control que no se actualiza). En E2 se evita porque la lectura se repite dentro del ciclo; si ocurriera, se detiene con el botón Terminate del IDE o con Ctrl + C en la terminal. También se ingresaron solo números, tal como indica la guía, porque la validación de tipo con `hasNextInt()` se verá más adelante.

6. **GRÁFICOS O FOTOGRAFÍAS:**

Las capturas de ejecución reproducen, con el aspecto de la terminal de VS Code, la salida real de cada programa al compilarlo y ejecutarlo con JDK 21 (lo que escribe el usuario aparece después de cada indicación). Para E3 se fijó temporalmente la semilla del generador (`new Random(7)`, `new Random(3)` y `new Random(2026)`) solo en copias de prueba, con el fin de conocer el número secreto y obtener evidencia reproducible; el programa entregado usa `new Random()`. La Figura 15 corresponde a una variante con 5 filas y 7 columnas obtenida cambiando las constantes.

E1 · Tabla de multiplicar

```
PS C:\Users\Cindy\Desktop\JAVA> javac Lab2_E1_Zambrano.java
PS C:\Users\Cindy\Desktop\JAVA> java Lab2_E1_Zambrano
TABLA DE MULTIPLICAR (10 x 10)
   1   2   3   4   5   6   7   8   9  10
   2   4   6   8  10  12  14  16  18  20
   3   6   9  12  15  18  21  24  27  30
   4   8  12  16  20  24  28  32  36  40
   5  10  15  20  25  30  35  40  45  50
   6  12  18  24  30  36  42  48  54  60
   7  14  21  28  35  42  49  56  63  70
   8  16  24  32  40  48  56  64  72  80
   9  18  27  36  45  54  63  72  81  90
  10  20  30  40  50  60  70  80  90 100
Instrucción interna ejecutada 100 veces.
PS C:\Users\Cindy\Desktop\JAVA>
```

Figura 14. *E1, ejecución 1: tabla completa de 10 × 10 y total de 100 ejecuciones.*

```
PS C:\Users\Cindy\Desktop\JAVA> javac Lab2_E1_Zambrano.java
PS C:\Users\Cindy\Desktop\JAVA> java Lab2_E1_Zambrano
TABLA DE MULTIPLICAR (5 x 7)
   1   2   3   4   5   6   7
   2   4   6   8  10  12  14
   3   6   9  12  15  18  21
   4   8  12  16  20  24  28
   5  10  15  20  25  30  35
Instrucción interna ejecutada 35 veces.
PS C:\Users\Cindy\Desktop\JAVA>
```

Figura 15. *E1, ejecución 2: variante con 5 filas y 7 columnas (35 ejecuciones).*

E2 · Estadística de un sensor

```
PS C:\Users\Cindy\Desktop\JAVA> javac Lab2_E2_Zambrano.java
PS C:\Users\Cindy\Desktop\JAVA> java Lab2_E2_Zambrano
Temperatura (valor negativo para terminar): 20.5
Temperatura (valor negativo para terminar): 22
Temperatura (valor negativo para terminar): 19.5
Temperatura (valor negativo para terminar): -1
Lecturas: 3 Promedio: 20.67 Max: 22.00 Min: 19.50
PS C:\Users\Cindy\Desktop\JAVA>
```

Figura 16. *E2, ejecución 1: datos de la guía (20.5, 22, 19.5, -1).*

```
PS C:\Users\Cindy\Desktop\JAVA> javac Lab2_E2_Zambrano.java
PS C:\Users\Cindy\Desktop\JAVA> java Lab2_E2_Zambrano
Temperatura (valor negativo para terminar): -5
No hubo lecturas válidas: no se muestran estadísticas.
PS C:\Users\Cindy\Desktop\JAVA>
```

Figura 17. *E2, ejecución 2: primera lectura negativa; no se divide para cero.*

```
PS C:\Users\Cindy\Desktop\JAVA> javac Lab2_E2_Zambrano.java
PS C:\Users\Cindy\Desktop\JAVA> java Lab2_E2_Zambrano
Temperatura (valor negativo para terminar): 0
Temperatura (valor negativo para terminar): 35.2
Temperatura (valor negativo para terminar): 18
Temperatura (valor negativo para terminar): 18
Temperatura (valor negativo para terminar): 27.75
Temperatura (valor negativo para terminar): -3
Lecturas: 5 Promedio: 19.79 Max: 35.20 Min: 0.00
PS C:\Users\Cindy\Desktop\JAVA>
```

Figura 18. *E2, ejecución 3 (caso propio): lectura 0 y valores repetidos.*

E3 · Adivina el número

```
PS C:\Users\Cindy\Desktop\JAVA> javac Lab2_E3_Zambrano.java
PS C:\Users\Cindy\Desktop\JAVA> java Lab2_E3_Zambrano
ADIVINA EL NÚMERO: pensé un número entre 1 y 100.
Tienes 7 intentos.
Intento 1 de 7: 50
  El número secreto es más bajo.
Intento 2 de 7: 25
  El número secreto es más alto.
Intento 3 de 7: 37
¡Acertaste en 3 intento(s)!
PS C:\Users\Cindy\Desktop\JAVA>
```

Figura 19. *E3, partida ganada en 3 intentos.*

```
PS C:\Users\Cindy\Desktop\JAVA> javac Lab2_E3_Zambrano.java
PS C:\Users\Cindy\Desktop\JAVA> java Lab2_E3_Zambrano
ADIVINA EL NÚMERO: pensé un número entre 1 y 100.
Tienes 7 intentos.
Intento 1 de 7: 10
  El número secreto es más alto.
Intento 2 de 7: 90
  El número secreto es más bajo.
Intento 3 de 7: 20
  El número secreto es más alto.
Intento 4 de 7: 80
  El número secreto es más bajo.
Intento 5 de 7: 30
  El número secreto es más alto.
Intento 6 de 7: 40
  El número secreto es más bajo.
Intento 7 de 7: 50
  El número secreto es más bajo.
Perdiste. El número era 35.
PS C:\Users\Cindy\Desktop\JAVA>
```

Figura 20. *E3, partida perdida tras 7 intentos: se revela el número secreto.*

```
PS C:\Users\Cindy\Desktop\JAVA> javac Lab2_E3_Zambrano.java
PS C:\Users\Cindy\Desktop\JAVA> java Lab2_E3_Zambrano
ADIVINA EL NÚMERO: pensé un número entre 1 y 100.
Tienes 7 intentos.
Intento 1 de 7: 50
  El número secreto es más alto.
Intento 2 de 7: 75
  El número secreto es más alto.
Intento 3 de 7: 88
  El número secreto es más alto.
Intento 4 de 7: 94
  El número secreto es más alto.
Intento 5 de 7: 97
  El número secreto es más alto.
Intento 6 de 7: 99
  El número secreto es más alto.
Intento 7 de 7: 100
¡Acertaste en 7 intento(s)!
PS C:\Users\Cindy\Desktop\JAVA>
```

Figura 21. *E3, acierto en el último intento permitido (intento 7 de 7).*

E4 · Rombo de asteriscos

```
PS C:\Users\Cindy\Desktop\JAVA> javac Lab2_E4_Zambrano.java
PS C:\Users\Cindy\Desktop\JAVA> java Lab2_E4_Zambrano
Ingrese un número impar positivo (tamaño del rombo): 4
  Valor inválido: debe ser impar y positivo.
Ingrese un número impar positivo (tamaño del rombo): 0
  Valor inválido: debe ser impar y positivo.
Ingrese un número impar positivo (tamaño del rombo): 5
   *
  ***
 *****
  ***
   *
PS C:\Users\Cindy\Desktop\JAVA>
```

Figura 22. *E4, ejecución 1: validación de 4 y 0 y rombo con n = 5.*

```
PS C:\Users\Cindy\Desktop\JAVA> javac Lab2_E4_Zambrano.java
PS C:\Users\Cindy\Desktop\JAVA> java Lab2_E4_Zambrano
Ingrese un número impar positivo (tamaño del rombo): -3
  Valor inválido: debe ser impar y positivo.
Ingrese un número impar positivo (tamaño del rombo): 9
    *
   ***
  *****
 *******
*********
 *******
  *****
   ***
    *
PS C:\Users\Cindy\Desktop\JAVA>
```

Figura 23. *E4, ejecución 2: validación de -3 y rombo con n = 9.*

E5 · Cajero automático

```
PS C:\Users\Cindy\Desktop\JAVA> javac Lab2_E5_Zambrano.java
PS C:\Users\Cindy\Desktop\JAVA> java Lab2_E5_Zambrano
--- CAJERO AUTOMÁTICO ---
1) Consultar  2) Depositar  3) Retirar  0) Salir
Opción: 1
Saldo: $500.00

--- CAJERO AUTOMÁTICO ---
1) Consultar  2) Depositar  3) Retirar  0) Salir
Opción: 2
Monto a depositar: 150
Depósito aceptado

--- CAJERO AUTOMÁTICO ---
1) Consultar  2) Depositar  3) Retirar  0) Salir
Opción: 3
Monto a retirar: 800
Fondos insuficientes o monto inválido

--- CAJERO AUTOMÁTICO ---
1) Consultar  2) Depositar  3) Retirar  0) Salir
Opción: 3
Monto a retirar: -20
Fondos insuficientes o monto inválido
```

Figura 24. *E5, secuencia de prueba de la guía (parte 1).*

```
--- CAJERO AUTOMÁTICO ---
1) Consultar  2) Depositar  3) Retirar  0) Salir
Opción: 3
Monto a retirar: 100
Retiro aceptado

--- CAJERO AUTOMÁTICO ---
1) Consultar  2) Depositar  3) Retirar  0) Salir
Opción: 1
Saldo: $550.00

--- CAJERO AUTOMÁTICO ---
1) Consultar  2) Depositar  3) Retirar  0) Salir
Opción: 9
Opción inválida

--- CAJERO AUTOMÁTICO ---
1) Consultar  2) Depositar  3) Retirar  0) Salir
Opción: 0
Fin del programa
PS C:\Users\Cindy\Desktop\JAVA>
```

Figura 25. *E5, secuencia de prueba de la guía (parte 2): saldo final $550.00.*

```
PS C:\Users\Cindy\Desktop\JAVA> javac Lab2_E5_Zambrano.java
PS C:\Users\Cindy\Desktop\JAVA> java Lab2_E5_Zambrano
--- CAJERO AUTOMÁTICO ---
1) Consultar  2) Depositar  3) Retirar  0) Salir
Opción: 3
Monto a retirar: 0
Fondos insuficientes o monto inválido

--- CAJERO AUTOMÁTICO ---
1) Consultar  2) Depositar  3) Retirar  0) Salir
Opción: 2
Monto a depositar: 0
Monto inválido

--- CAJERO AUTOMÁTICO ---
1) Consultar  2) Depositar  3) Retirar  0) Salir
Opción: 3
Monto a retirar: 500
Retiro aceptado

--- CAJERO AUTOMÁTICO ---
1) Consultar  2) Depositar  3) Retirar  0) Salir
Opción: 3
Monto a retirar: 1
Fondos insuficientes o monto inválido
```

Figura 26. *E5, caso propio con montos en cero y retiro exacto (parte 1).*

```
--- CAJERO AUTOMÁTICO ---
1) Consultar  2) Depositar  3) Retirar  0) Salir
Opción: 1
Saldo: $0.00

--- CAJERO AUTOMÁTICO ---
1) Consultar  2) Depositar  3) Retirar  0) Salir
Opción: 7
Opción inválida

--- CAJERO AUTOMÁTICO ---
1) Consultar  2) Depositar  3) Retirar  0) Salir
Opción: 0
Fin del programa
PS C:\Users\Cindy\Desktop\JAVA>
```

Figura 27. *E5, caso propio (parte 2): saldo $0.00 y opción inválida.*

Errores reproducidos (sección 5.5)

```
PS C:\Users\Cindy\Desktop\JAVA> java Lab2_E2_Zambrano
Temperatura (valor negativo para terminar): 20.5
Temperatura (valor negativo para terminar): 22
Temperatura (valor negativo para terminar): 19.5
Temperatura (valor negativo para terminar): -1
Lecturas: 3 Promedio: 20.67 Max: 22.00 Min: 0.00
PS C:\Users\Cindy\Desktop\JAVA>
```

Figura 28. *Error 1: mínimo inicializado en 0.*

```
PS C:\Users\Cindy\Desktop\JAVA> java Lab2_E2_Zambrano
Temperatura (valor negativo para terminar): -5
Lecturas: 0 Promedio: NaN Max: 0.00 Min: 0.00
PS C:\Users\Cindy\Desktop\JAVA>
```

Figura 29. *Error 2: promedio NaN al no controlar cantidad = 0.*

```
PS C:\Users\Cindy\Desktop\JAVA> java DivCero
Exception in thread "main" java.lang.ArithmeticException: / by zero
at DivCero.main(DivCero.java:1)
PS C:\Users\Cindy\Desktop\JAVA>
```

Figura 30. *Error 3: ArithmeticException con división entera para cero.*

```
PS C:\Users\Cindy\Desktop\JAVA> java Lab2_E5_Zambrano

--- CAJERO AUTOMÁTICO ---
1) Consultar  2) Depositar  3) Retirar  0) Salir
Opción: 1
Saldo: $500.00
Monto a depositar: 100
Depósito aceptado

--- CAJERO AUTOMÁTICO ---
1) Consultar  2) Depositar  3) Retirar  0) Salir
Opción: 0
Fin del programa
PS C:\Users\Cindy\Desktop\JAVA>
```

Figura 31. *Error 4: falta de break en case 1.*

```
PS C:\Users\Cindy\Desktop\JAVA> java Lab2_E3_Zambrano
ADIVINA EL NÚMERO: pensé un número entre 1 y 100.
Tienes 7 intentos.
Intento 1 de 7: 10
  El número secreto es más alto.
Intento 2 de 7: 90
  El número secreto es más bajo.
Intento 3 de 7: 20
  El número secreto es más alto.
Intento 4 de 7: 80
  El número secreto es más bajo.
Intento 5 de 7: 30
  El número secreto es más alto.
Intento 6 de 7: 40
  El número secreto es más bajo.
Intento 7 de 7: 50
  El número secreto es más bajo.
Intento 8 de 7: 60
  El número secreto es más bajo.
Perdiste. El número era 35.
PS C:\Users\Cindy\Desktop\JAVA>
```

Figura 32. *Error 5: condición con <= que permite un octavo intento.*

7. **DISCUSIÓN:**

Los resultados confirman la teoría de la unidad: cuando el número de repeticiones se conoce (E1 y las filas del rombo) el for es la herramienta más clara, porque inicialización, condición y actualización quedan juntas; cuando la repetición depende de un dato que llega durante la ejecución (E2) el while con centinela permite incluso no entrar al ciclo; y cuando el bloque debe ejecutarse al menos una vez (E3, validación de E4 y menú de E5) el do-while evita duplicar código.

En E1, el conteo de 100 ejecuciones y de 35 en la variante 5 × 7 coincide con el análisis $n \cdot m$: el ciclo interno completa todas sus vueltas por cada vuelta del externo. En E2, las dos formas de error reproducidas (NaN con double y ArithmeticException con enteros) muestran que el caso «sin datos» no es un detalle: sin el if (cantidad > 0) el programa produce resultados incorrectos aunque compile sin problemas, y la inicialización del mínimo con la primera lectura evita un mínimo falso.

En E3 la condición compuesta del do-while (no acertó y quedan intentos) hace que el programa termine por dos causas distintas; por eso se probaron la partida ganada, la perdida y el acierto en el último intento, que son los límites de la condición. El peor caso de 7 intentos coincide con el cálculo teórico $\lfloor \log_2(101) \rfloor = 7$. En E4 y E5, la validación con do-while y las decisiones anidadas impidieron rombos inválidos, retiros mayores al saldo y montos no positivos, y el switch con break evitó la ejecución de casos adicionales.

Correspondencia con la rúbrica de evaluación

Criterio	Pts	Cómo se cumple	Evidencia
Funcionamiento E1–E5	6	Los cinco programas compilan sin advertencias y todas las salidas coinciden con lo esperado.	Sección 5.1 y Figuras 1–27
Uso adecuado de ciclos	2	Cada ejercicio usa el ciclo que corresponde al criterio de la guía y se justifica.	Secciones 3.2 y 5.4 (pregunta 1)
Validación y pruebas	1	Se validan entradas (E4, E5) y se prueban casos límite: sin lecturas, un solo dato, último intento, n = 1, montos en cero y retiro exacto.	Secciones 5.1 y 5.5
Informe	1	Incluye análisis, pseudocódigo, código comentado, capturas, pruebas de escritorio, respuestas, errores, conclusiones y registro de IA.	Todo el informe

8. **CONCLUSIONES:**

- Elegir el ciclo según el problema (for con cantidad conocida, while con centinela y do-while para validar o repetir menús) hizo que cada programa fuera más corto, claro y fácil de comprobar.
- Los ciclos anidados repiten la instrucción interna $n \cdot m$ veces; contar las ejecuciones (100 en 10 × 10) permitió verificar el análisis teórico y comprender cómo se construyen tablas y figuras como el rombo.
- Los casos límite revelan errores que no aparecen con datos normales: sin una validación como `cantidad > 0`, E2 muestra NaN o falla con ArithmeticException, y un mínimo mal inicializado nunca se actualiza.
- Una condición de salida bien definida y una variable de control que se actualiza dentro del ciclo evitan ciclos infinitos; en E3 la condición compuesta garantizó el límite de 7 intentos.
- Combinar do-while con switch y decisiones anidadas permitió un cajero que rechaza montos inválidos y sobregiros, y mostró la importancia de poner break en cada case.

Recomendaciones

- Definir antes de programar la condición de salida de cada ciclo y verificar que se alcanza.
- Inicializar contadores y acumuladores justo antes del ciclo que los utiliza, y usar la primera lectura válida para máximos y mínimos.
- Probar siempre los casos límite: ninguna repetición, una sola repetición y el último intento permitido.
- Mantener una sangría consistente en estructuras anidadas para identificar a qué bloque pertenece cada instrucción.
- Detener un ciclo infinito con el botón Terminate del IDE y revisar la actualización de la variable de control.
- Verificar las sugerencias de herramientas de IA compilando y ejecutando el código con los datos de prueba.

9. **BIBLIOGRAFÍA:**

Oracle. 2023. Java Platform, Standard Edition 21 API Specification, Oracle, https://docs.oracle.com/en/java/javase/21/docs/api/. Fecha de consulta: 29 de octubre de 2026.

Oracle. s. f. The Java Tutorials: Control Flow Statements (while, do-while y for), Oracle, https://docs.oracle.com/javase/tutorial/java/nutsandbolts/flow.html. Fecha de consulta: 29 de octubre de 2026.

Oracle. 2023. Class Formatter (formato de printf), Java SE 21 API, https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/util/Formatter.html. Fecha de consulta: 29 de octubre de 2026.

Universidad de las Fuerzas Armadas ESPE. 2026. Guía de laboratorio 2: Estructuras repetitivas y anidamiento de estructuras de control en Java (VDC-GUI-2024-V2-015), Fundamentos de Programación, NRC 37712, Departamento de Ciencias de la Computación, Sede Santo Domingo.

Gonzabay Jimenez, E. 2026. Presentaciones de la Unidad 1, Fundamentos de Programación, aula virtual ESPE.